Niki Bell

Vertex Venue: A Complete Software Delivery Plan

1) System Overview

The core problem Vertex Venue addresses is the chaotic, all at once, nature of the live music industry. Unlike a typical retail site where traffic is relatively steady, a concert venue platform experiences extreme surges. When a popular artist is announced, the system goes from idle to thousands of requests per second in an instant. This burst of traffic can easily crush a traditional server setup.

Vertex Venue is a web-based ticketing and event discovery platform. Its primary goal is to provide a seamless, fair, and secure experience for fans while giving venue staff the tools they need to manage seating and sales.

Key Stakeholders

- **Concert-Goers (Users):** They need the site to be fast and stable. If the site lags while they are trying to buy tickets, they lose out, and the venue loses trust.
- **Developers:** They need to push updates (like new interactive maps) without breaking the live ticketing engine.
- **Operations & Security:** They are responsible for the plumbing: ensuring the site scales automatically and that user payment data remains encrypted and safe from hackers.

---

2) Delivery Pipeline Design

In DevOps, The First Way is all about creating a smooth, fast flow of work from Development to Operations. For Vertex Venue, we replace manual hand-offs with a fully automated CI/CD pipeline.

How Code Moves:

When a developer finishes a new feature, they commit their code to GitHub. This acts as the single source of truth. From there, the pipeline takes over:

1. **Lint & Test Stage**: Automated checks and unit tests (like Jest) run immediately on the raw code. If a new piece of code breaks the checkout process, the pipeline fails instantly, and the code never reaches the live site. This ensures a fast feedback loop.

2. Build Stage: Once the code passes all tests, we use Docker to package the application. By containerizing the approved code, we ensure that the app runs exactly the same way on a developer's laptop as it does on a production server. This eliminates the "it worked on my machine" problem, as we have learned from the textbook.

3. Push & Release: The finished Docker image is pushed to a private registry where it is securely stored and prepared for deployment to the cloud. All must pass to be pushed.

Automation vs. Manual Work

In older systems, an engineer might have to manually log into a server and upload files. In our system, automation replaces every manual touchpoint. We use Infrastructure as Code (IaC) tools like **OpenTofu** to provision our servers. If we need ten more servers to handle a sold-out show, the code does it for us automatically. This reduces human error and ensures the system is repeatable.

---

3) Environments & Infrastructure

To keep Vertex Venue safe, we never test in production. Instead, we use three distinct environments that act as safety nets.

- **Development:** This is the playground to try things out. Developers can experiment with new UI designs or search algorithms here without any risk to actual ticket sales.
- **Staging:** This is the most critical environment. It is a mirror image of the production site. Before a big release, we run load tests in staging to see exactly how many users the system can handle before it slows down. Using our staging environment correctly is what brings us success in production, therefore it is a crucial step.
- **Production:** The live environment where real money and real tickets live. This is what our users see and interact with.

Release Strategy: Blue-Green Deployment

We use a **Blue-Green strategy** to ensure zero downtime. We will have two identical versions of Vertex Venue running. "Blue" is live. When we have an update, we deploy it to "Green." Once we verify Green is working perfectly, we flip a virtual switch (at the load balancer level) to send all fans to Green. If anything goes wrong, we flip the switch back to Blue in milliseconds.

This infrastructure is cloud-native, meaning we don't own physical servers. We use managed services like AWS or Google Cloud, which allow us to rent more power exactly when a concert goes on sale and turn it off when the surge is over. This is crucial for this website since we need more power support in the bursts of traffic we often experience.

---

4) Networking, Security, and Data

This is the internal architecture that keeps the platform honest and safe.

Networking & Communication

Vertex Venue uses a **Virtual Private Cloud (VPC)** to create a digital fence around our system.

- **Public Subnets:** Only the Load balancer and a Content Delivery Network (CDN) are exposed to the public internet. This speeds up the site for fans by storing images of the venue closer to their physical location.

- **Private Subnets:** Our application servers and databases are hidden in private subnets. They cannot be reached directly from the internet, which drastically minimizes the system's vulnerability to external threats.

Data Storage & Protection

For ticketing, we use a Relational Database (PostgreSQL via RDS). Ticketing requires ACID compliance, which is a fancy way of saying the database is perfect at math. It ensures that if two people click "Buy" on the same seat at the same time, the system only processes one transaction.

To protect this data, we use:

- **Encryption at Rest:** All data stored on disks is encrypted using AES-256.
- **Encryption in Transit:** We force TLS 1.3 for all traffic, ensuring that user passwords and credit card tokens are never sent in plain text.
- **Secrets Management:** We will never hardcode passwords. We use a Secrets Manager to inject credentials into the app only when it’s running.

---

5) Monitoring, Feedback, and Reliability:

The Second Way is about creating fast feedback loops. If Vertex Venue is struggling, we need to know before the users do.

Telemetry: The Four Golden Signals

We collect four specific signals to monitor system health:

1. **Latency:** How long does it take for a ticket map to load?
2. **Traffic:** How many people are on the site right now?
3. **Errors:** Is the "Buy" button failing for some users?
4. **Saturation:** Is our database running out of memory?

Observability & Feedback

By using **centralized logging** (like CloudWatch), we can interrogate the system. For example,if a user in Ogden, Utah, has a failed checkout, we can look at the logs and see exactly which microservice caused the error. This feedback allows us to set an Error Budget. We know no system is 100% perfect, so we balance our need to move fast with our need to stay stable based on the data.

---

6) Learning and Improvement: The Third Way

The Third Way is about building a culture of curiosity and growth. At Vertex Venue, we treat every mistake as an opportunity to get stronger, in both our teams and our system.

Continual Learning & Experimentation

We don't wait for things to break. We use Chaos Engineering where we purposely break a small part of the staging environment to see if the system auto-heals. For example, if we kill one server, does the load balancer automatically move traffic to a healthy one? This is

important to test out our system to see how it behaves in certain circumstances so we can help it be strong and prepared in those instances when they actually occur.

Blameless Postmortems

When a real failure happens (like maybe a 10-minute outage during a ticket drop), we hold a blameless postmortem. We don't ask "Who broke it?" We ask "What part of our process allowed this to happen?" We then use Retrieval-Augmented Generation (RAG) or documentation tools to share that lesson with the whole team. This turns a local discovery into a global improvement, ensuring that the same mistake never happens twice. This environment we create is very important to our company as we want all our workers to feel comfortable and like they can grow and learn without punishment or shame of their failures. Mistakes are inevitable, and so we pride ourselves in having a positive work space where we can learn and grow by spotting the system issue without blame and thus strengthening each team member and our system as a whole.

Conclusion

By integrating flow, feedback, and learning, Vertex Venue isn't just a website, it's a resilient delivery system. Instead of just building a product and waiting for issues to appear, we now use continuous feedback to make the system smarter and more resilient. This ensures that when the next big tour is announced, Vertex Venue is ready for the crowd.